



What is a Tool in LangChain?

Give your AI the power to DO things, not just TALK about things



Real-time Data



API Access



Custom Actions

✗ AI Without Tools

 User

What is today's date?

 AI

I don't have access to real-time data. My knowledge was last updated in **January 2024...** ✗

(AI has no access to current date!)

AI has **no real-time access** — it cannot know the current date!

✓ AI With Tools

User

What is today's date?



Calls `get_current_date` tool

AI

Today's date is **December 3, 2024** ✓

(Tool returns real-time data!)

Tools give AI **real-time access** to live data!



What is a Tool?

A **Tool** is a Python function that an AI agent can **call** to perform actions like getting live data, API calls, database queries, and more.



Current Date



Weather API



Web Search



Database



3 Parts of a Tool

1

Name

What the tool is called → `get_current_date`

2

Description

What the tool does → AI reads this to decide when to use it

3

Arguments

What inputs it needs → `None (no input required)`

⚡ How Tools Work

1



User asks: "What is today's date?"



2



AI reads tool descriptions



3



AI calls: `get_current_date()`



4



Tool returns: "December 3, 2024"



5



AI answers: "Today is December 3, 2024"

1

Install & Import

First, install langchain and import the tool decorator. This decorator will convert your normal Python function into a LangChain tool.

```
# Install: pip install langchain  
  
from langchain.tools import tool
```



The `@tool` decorator is all you need to create custom tools!

2

Define Function

Start by writing a simple Python function. Right now it's just a normal function, not a tool yet. We will convert it in the next steps.

```
def get_current_date():  
    return "December 3, 2024"
```



This is a regular Python function — AI can't use it yet!

3

Add Docstring

A docstring tells the AI what your tool does. Based on this description, the AI decides when to use your tool. Write it clearly!

```
def get_current_date():  
    """Get today's current date. Use this when someone asks for  
    today's date."""  
    return "December 3, 2024"
```



AI reads this docstring to understand when to call your function!

4

Add Type Hints

Type hints tell the AI what kind of data your function expects and returns. Without them, AI won't know what format to send data in.

```
def get_current_date() → str:  
    """Get today's current date."""  
    return "December 3, 2024"
```



→ str means this function returns a string (text)

5

Add @tool Decorator

Now add `@tool` above your function to convert it into a LangChain tool. The AI can now discover and call this tool automatically!

```
@tool
def get_current_date() → str:
    """Get today's current date."""
    return datetime.now().strftime("%B %d, %Y")
```

✨ Your function is now a Tool with name, description, and args!

6

View Tool Properties

Once you add `@tool`, your function gets new properties. AI uses these to understand how to call your tool.

```
print(get_current_date.name)
print(get_current_date.description)
print(get_current_date.args)
```

Output:

```
get_current_date
Get today's current date.
{}
```

7

Run the Tool

After adding `@tool`, you cannot call the function directly. Instead, use `.invoke()` — this is how AI calls your tool internally.

```
# ❌ Wrong: get_current_date()
```

```
# ✅ Correct:
```

```
result = get_current_date.invoke({})  
print(result)
```

Output:

December 3, 2024

Key Takeaways

- ✓ Tools give AI real-time access to live data
- ✓ Use @tool decorator to create tools
- ✓ Add docstring so AI knows when to use it
- ✓ Add type hints for input/output
- ✓ Use .invoke() to run the tool

Save this post and **start building!** 